

Consegna 6a - MPI

La consegna 6a riguarda l'apprendimento dei fondamenti della programmazione parallela utilizzando il modello MPI (Message Passing Interface) su sistemi HPC. Gli obiettivi sono: comprendere il funzionamento di mpirun e mpicc, acquisire familiarità con le primitive di comunicazione MPI (Send, Recv, Isend, Irecv, Bcast, Scatter, Gather, Reduce), analizzare gli schemi di comunicazione per le applicazioni cpi, heat e factorize, ed eseguire programmi MPI su PC personale e cluster HPC con integrazione Slurm.

Esercizi di base con mpirun e compilazione

Il comando `mpirun` è lo strumento per la creazione e l'esecuzione dei processi MPI. La distribuzione dei processi sui nodi è determinata da specifiche opzioni passate a linea di comando.

```
andre@andrea:base$ mpirun -n 4 date
Sun May 24 17:27:11 CEST 2026
Sun May 24 17:27:11 CEST 2026
Sun May 24 17:27:11 CEST 2026
Sun May 24 17:27:11 CEST 2026
```

Utilizzando l'opzione `-n` che definisce il numero di processi da attivare, il comando `date` è stato eseguito 4 volte in parallelo.

```
root@wn1:~# mpirun -host wn1:4,wn2:2 -n 5 hostname
wn1
wn1
wn1
wn2
wn1
```

Il flag `-host` serve a specificare la lista dei nodi per distribuire i processi su più worker nodes.

Per poter sviluppare codice, abbiamo utilizzato `mpicc`, ovvero il wrapper MPI del compilatore standard, corrisponde a `gcc` e gestisce l'inclusione di MPI in automatico.

- `mpi1_helloworld`: verifica l'inizializzazione dell'ambiente (`MPI_Init`), il recupero del numero totale di task e del proprio rank.

```
andre@andrea:base$ mpirun -n 4 ./mpi1_helloworld
Hello from task 2 on andrea!
Hello from task 3 on andrea!
Hello from task 1 on andrea!
Hello from task 0 on andrea!
MASTER: Number of MPI tasks is: 4
```

- `mpi2a_send-recv.c`: comunicazione bloccante punto-punto con `MPI_Send()` e `MPI_Recv()`. Possibile deadlock se non ordinato correttamente.

```
andre@andrea:base$ mpirun -n 4 ./mpi2a_send-recv
Process 1 receives 10 from process 0.
Process 0 sends 10 to process 1.
```

- `mpi2b_send_Irecv.c` : comunicazione asincrona con `MPI_Irecv()` non bloccante, permettendo overlap computation-communication.

```
andre@andrea:base$ mpirun -n 4 mpi2b_send-Irecv
Process 0 wait 4 seconds before sending
Data not ready: receiver sleep 1 sec
Data not ready: receiver sleep 1 sec
Data not ready: receiver sleep 1 sec
Data not ready: receiver sleep 1 sec
Process 0 sends 10 to process 1
Process 1 receives 10 from process 0
```

- `mpi3_send_recv-array.c` : comunicazione di strutture dati complesse (array). Fondamentale per heat e factorize.

```
andre@andrea:base$ mpirun -n 4 mpi3_send_recv-array
Process 0 sends the following array to process 1.
  0.00  1.00  2.00  3.00  4.00  5.00  6.00  7.00  8.00  9.00
Process 1 receives the following array from process 0.
  0.00  1.00  2.00  3.00  4.00  5.00  6.00  7.00  8.00  9.00
```

- `mpi4_sendrecv.c` : comunicazione bidirezionale con `MPI_Sendrecv()` simultaneo, eliminando deadlock.

```
andre@andrea:base$ mpirun -n 4 ./mpi4_sendrecv
Process 0/4 receives 3 from process 3
Process 1/4 receives 0 from process 0
Process 3/4 receives 2 from process 2
Process 2/4 receives 1 from process 1
```

- `mpi5_ring.c` : topologia ring con comunicazione tra vicini. Importante per simulazioni iterate come heat.

```
andre@andrea:base$ mpirun -n 4 ./mpi5_ring
Ogni task determina il rank del task che precede (prev) e che segue (next) - MPIsize= 4
Rank 2 - Sum 6
Rank 3 - Sum 6
Rank 0 - Sum 6
Rank 1 - Sum 6
```

- `mpi7_bcast.c` : operazione collettiva `MPI_Bcast()` che distribuisce dati da root a tutti i task. Essenziale per sincronizzare parametri globali in cpi.

```
andre@andrea:base$ mpirun -n 4 ./mpi7_bcast
[0]: Before Bcast, buf is 777
```

```
[2]: Before Bcast, buf is 24
[3]: Before Bcast, buf is 24
[0]: After Bcast, buf is 777
[2]: After Bcast, buf is 777
[1]: Before Bcast, buf is 24
[3]: After Bcast, buf is 777
[1]: After Bcast, buf is 777
```

- `mpi8_scatter_gather.c`: `MPI_Scatter()` distribuisce porzioni di array ai 4 task, ogni task incrementa, `MPI_Gather()` raccoglie i risultati,

```
andre@andrea:base$ mpirun -n 4 ./mpi8_scatter_gather
mpi_rank: 1, mpi_size 4
mpi_rank: 2, mpi_size 4
mpi_rank: 3, mpi_size 4
mpi_rank: 0, mpi_size 4
0 1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17 18 19 20 21 22 23 24 25 26 27 28 29 30
31 32 33 34 35 36 37 38 39
Ogni task riceve dati con MPI_scatter(). incrementa gli interi ricevuti e rispedisc
e con MPI_gather()
1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17 18 19 20 21 22 23 24 25 26 27 28 29 30 31
32 33 34 35 36 37 38 39 40
```

- `mpi9_reduction.c`: `MPI_Reduce()` applica operazione aggregante (somma) su dati di tutti i task. Cruciale per calcoli globali in cpi.

```
andre@andrea:base$ mpirun -n 4 ./mpi9_reduction
tramite MPI_Reduce() ogni task invia il proprio rank, il rank 0 riceve e somma
mpi_size: 4, reduction: 6
```

- `mpi_DataMining.c`: applicazione complessa combinando Send bloccante, Irecv non-bloccante, Scatter. Ogni task processa 128 elementi parallelamente. Quando un task trova il massimo, lo comunica agli altri.

```
andre@andrea:base$ mpicc mpi_DataMining.c -o mpi_DataMining
andre@andrea:base$ mpirun -n 4 ./mpi_DataMining
Task 0/4 domain 128
Task 1/4 domain 128
Task 2/4 domain 128
Task 3/4 domain 128
Task 1/4 MPI_Irecv started
task 1 processing 1
Task 2/4 MPI_Irecv started
task 2 processing 2
Task 3/4 MPI_Irecv started
task 3 processing 3
Task 0/4 MPI_Irecv started
task 0 processing 0
[...]
```

```
task 1 processing 121
Task 1 found 121
task 3 processing 123
Task 1/4 - RECEIVED 121 from 1, exiting
Task 2/4 - RECEIVED 121 from 1, exiting
Task 0/4 - RECEIVED 121 from 1, exiting
Task 3/4 - RECEIVED 121 from 1, exiting
```

Analisi primitive MPI per CPI, Heat e Factorize

- **Primitive CPI:**

- `MPI_Bcast()` necessaria per il processo master per inviare a tutti i worker il numero totale di intervalli da utilizzare per l'integrazione numerica.
- `MPI_Reduce()` al termine del calcolo, ogni worker avrà un'area parziale della curva, con l'operatore `MPI_SUM`, il rank 0 raccoglierà e sommerà automaticamente tutte queste aree parziale per ottenere il valore finale di π .

- **Primitive Heat:**

- `MPI_Scatter` / `MPI_Gather` : utili in fase di setup per distribuire le righe della matrice ai vari processi e poi raccoglierle al termine della simulazione per l'output.
- `MPI_Sendrecv()` : primitiva per lo scambio delle ghost cells. Ad ogni timestep della simulazione, le celle di bordo del sottodominio di un task devono essere inviate ai task vicini. ricevendo contemporaneamente le loro. `Sendrecv` previene i deadlock in queste operazioni cicliche.

- **Primitive Factorize:**

- `MPI_Bcast()` : per comunicare il numero da fattorizzare a tutti i worker.
- `MPI_Irecv` / `MPI_Test` : trattandosi di una ricerca, appena un task trova i fattori corretti, deve poter comunicare agli altri di fermarsi per sprecare risorse. L'utilizzo di comunicazioni *non bloccanti* permette ai task di calcolare e, ad ogni iterazioni, fare un rapido "test" per controllare se un altro processo ha già inviato il segnale di stop.

Esecuzione via SLURM

Sul cluster HPC, l'assegnazione delle risorse deve passare tramite le code gestite da Slurm. Slurm si integra nativamente con openMPI: non è necessario specificare a `mpirun` parametri come `-n` o `-host`, poiché queste informazioni (numero di task, configurazione dei nodi) vengono ereditate automaticamente dalle direttive `#SBATCH` impostate nello script.

```
mpirun_hostname.slurm
```

Questo script esegue il comando base `hostname` per verificare l'allocazione dei nodi.

- Le variabili d'ambiente mostrano che SLURM ha assegnato 2 nodi specifici, ovvero `wn20` e `wn99`.
- Sono stati richiesti 4 task totali, distribuiti come 2 CPU per ogni nodo.

```
#SLURM_JOB_NODELIST      : wn[20,99] +
#SLURM_NNODES            : 2
#SLURM_NTASKS            : 4
```

```
#SLURM_JOB_CPUS_PER_NODE : 2(x2)
wn20
wn99
wn99
wn20
```

mpi1_helloworld.slurm

Stampa il rank del processo e il nodo su cui si trova. Le risorse allocate sono i nodi `wn[20,99]` con 4 task e 2 core per nodo. I task 0 e 1 vengono eseguiti dal nodo `wn20` e i task 2 e 3 ospitati dal nodo `wn99`.

```
module
#SLURM_JOB_NODELIST      : wn[20,99]
#SLURM_NNODES            : 2
#SLURM_NTASKS            : 4
#SLURM_JOB_CPUS_PER_NODE : 2(x2)
Hello from task 0 on wn20!
Hello from task 1 on wn20!
MASTER: Number of MPI tasks is: 4
Hello from task 2 on wn99!
Hello from task 3 on wn99!
```

mpi_DataMining.slurm

L'algoritmo di ricerca viene istanziato su due nodi fisici differenti (`wn[20-21]`). Il task 2 trova l'elemento desiderato e, grazie al design asincrono con `MPI_Irecv`, notifica istantaneamente gli altri task portando il programma a una terminazione coordinata e pulita:

```
#SLURM_JOB_NODELIST      : wn[20-21]
#SLURM_NNODES            : 2
#SLURM_NTASKS            : 4
#SLURM_JOB_CPUS_PER_NODE : 2(x2)
Task 0/4 domain 128
[...]
Task 2 found 110
Task 1/4 - RECEIVED 110 from 2, exiting
Task 0/4 - RECEIVED 110 from 2, exiting
Task 3/4 - RECEIVED 110 from 2, exiting
Task 2/4 - RECEIVED 110 from 2, exiting
```